Part D — Code Refactoring and System Optimization

FEATURE	BEFORE	PROBLEM IDENTIFIED	AFTER	JUSTIFICATION
1.Dynamic memory resizing	int* temp = new int[maxCapacity * 2]; for (...) temp[i] = dataArray[i]; dataArray = temp;	Old dataArray was not released before the pointer was replaced, causing a heap memory leak on every resize.	int* temp = new int[newCapacity]; for (size_t i = 0; i < size; ++i) temp[i] = data[i]; delete[] data; data = temp; capacity = newCapacity;	Releases the old buffer before transferring the new allocation, preventing the resize leak.
2. Array deletion shifting	Nested loop inside the deletion shift; the inner loop breaks after one copy.	The nested structure is redundant and makes the deletion logic harder to read.	for (size_t i = static_cast(index); i < size - 1; ++i) data[i] = data[i + 1];--size;	Uses a clear single-pass shift. The operation remains O(n) but removes unnecessary loop overhead.
3. Global state / encapsulation	Global dataArray, currentCount, maxCapacity.	Mutable global state reduces modularity and makes multiple independent array instances difficult.	class DynamicArray { private: int* data; size_t size; size_t capacity; }	Encapsulates data and behavior in a class and removes the mutable global array state
4. Bounds validation	remove_item_at(i nt idx) did not reject idx >= currentCount.	Invalid upper-bound indices could access memory outside the valid array range.	if (index < 0 || static_cast(index) >= size) { cout << "Invalid index!"; return false; }	Enforces the required valid range 0 ≤ index < size and safely rejects invalid requests.

5. Array display bounds	for (int i = 0; i <= currentCount; ++i)	The <= condition reads one position beyond the last valid element.	for (size_t i = 0; i < size; ++i) cout << data[i] << " ";	Only valid elements from index 0 through size-1 are displayed.
6. Memory lifecycle	No delete[] dataArray before program exit.	Heap memory remained allocated when the legacy	~DynamicArray( ) { delete[] data; data = nullptr; }	RAII-style cleanup releases the heap allocation automatically when

		program ended.		the object goes out of scope.
7. Naming / code style	Mixed names such as add_item, findItem, and printAll.	Inconsistent naming reduces readability and makes the API less uniform.	addItem(), removeItemAt (), findItem(), printAll(), getSize(), getCapacity()	Consistent camelCase naming improves readability and makes the class interface easier to understand.
8. Copy safety	Raw pointer could be copied shallowly in a class-based design	Shallow copying a raw owning pointer could lead to double deletion or shared ownership problems.	DynamicArray(co nst DynamicArray&) = delete; DynamicArray& operator=(const DynamicArray&) = delete;	Prevents accidental shallow copying of the owning raw pointer